

Software Engineering

A practical study reference with foundational concepts, examples, and common review questions.

1. Requirements and Lifecycle

Software engineering uses disciplined practices to build and maintain software. Lifecycle activities include discovery, requirements, design, implementation, testing, deployment, and maintenance; teams revisit them as they learn.

Functional requirements describe behavior; nonfunctional requirements describe qualities such as performance, security, availability, accessibility, and maintainability. Acceptance criteria make outcomes testable. Clarify edge cases and failure behavior early.

2. Design and Implementation

Architecture defines components, responsibilities, and communication. High cohesion and low coupling support change. Common styles include layered, client-server, event-driven, and services; each adds trade-offs. Prefer the simplest design that meets requirements.

Readable code uses clear names, focused functions, explicit errors, and tests. Version control records changes; coherent commits and reviews improve traceability. Continuous integration runs repeatable checks such as builds, tests, and linting.

3. Testing and Delivery

Unit tests cover small units; integration tests cover boundaries; end-to-end tests cover user flows. Test expected cases, boundaries, invalid input, and recovery. Static analysis, security review, accessibility, and load tests complement automated tests.

Agile emphasizes iterative delivery and feedback. Scrum and Kanban are frameworks, not goals. Technical debt is future cost caused by shortcuts or constraints; track its impact and plan. Safe releases need traceability, monitoring, and rollback.

4. Common Questions

Verification asks whether the product meets specified requirements; validation asks whether it solves the user's actual problem. Requirements align stakeholders and guide design and tests. Code review should focus on correctness, security, clarity, and maintainability.